Open Engineering Architecture & Agentic Systems Series

From Vibe Coding to Agentic Engineering

Student Textbook & Reference Manual: Vibe Coding Foundations, Context Physics & Harness Engineering (agy & claude)

Author: Alejandro Kaspar — AI Forward Deployed Engineer · Licensed under CC BY 4.0 & Apache 2.0

How to Read This Book & Complete the Hands-On Labs (Student Guide)

Welcome to *From Vibe Coding to Agentic Engineering*. This textbook is organized as a two-phase engineering progression paired with eight self-diagnosing Python laboratories (`labs/lab_01_*` through `labs/lab_08_*`). Every chapter develops the core theory from first principles, illustrates the system mechanics with visual blueprints, and concludes with a step-by-step lab guide that you can complete on either the **Google Antigravity (`agy` branch)** or **Anthropic Claude Code (`claude` branch)** track:

- **Chapter 1 — Foundations of Natural Language Programming & The Agentic Loop (Modules 1–2, Labs 01–02):** Why source code is an externalized model of domain understanding, how natural language coordinates executable actions, the 2022–2026 architectural evolution of coding agents, the four-pillar engineering skills framework, and HTTP/JSONL wire-trace analysis of the core agentic loop.
- **Chapter 2 — The `RePPIT` Workflow, Context Window Physics & Intentional Compaction (Modules 3–4, Labs 03–04):** Matching workflow ceremony to task complexity, defeating silent assumptions through Socratic requirement interviews, generating orthogonal architectural proposals with post-proposal context resets, understanding the 40% "Dumb Zone" phase transition, and practicing Frequent Intentional Compaction (`RPI`).
- **Chapter 3 — Harness Engineering Layer 1: Context Primitives, Rules & Persistent Memory (Module 5, Lab 05):** The cybernetic harness equation (Agent = Model + Harness), the 2×2 control matrix of feedforward guides and feedback sensors, the 150-instruction compliance cliff, the minimalist 60-line root map rule (`CLAUDE.md` / `GEMINI.md` / `AGENTS.md`), and cross-session auto-memory with epistemic provenance tags.
- **Chapter 4 — Harness Engineering Layer 2: Progressive-Disclosure Skills, API Grounding & Meta-MCP (Module 6, Lab 06):** Three-level progressive disclosure (`SKILL.md`), version-pinned API documentation with self-improving session annotations (`chub`), ergonomic MCP tool design and Meta-MCP "Code Mode" (> 90% token savings), empirical skill benchmarks, and Signal Detection Theory (d', c) trigger calibration.
- **Chapter 5 — Multi-Agent Subagent Firewalls, Governance Hooks, Autonomous Ratchets & Program Design (Modules 7–8, Labs 07–08):** Subagents as context firewalls (50k → 500 tokens), multi-agent pull-request triage, anonymized cross-model peer councils, parallel workspace isolation (`git worktree` vs. colocated Jujutsu `jj`), four-tier governance hooks, remediation linters, durable execution, bounded autonomous research ratchets, and front-loaded program design.

Contents

1 Phase 1 (Modules 1 & 2): Natural Language to Code, The 2022–2026 Timeline & The Agentic Loop (agy & claude 101) **3**

1.1 Why Natural Language Correlates with Code, Language Philosophy & Software Engineering Fundamentals . . 3

1.2 The 2021–2026 Architectural Evolution, The 17-Day Flagship Cadence & Model-Agnostic Harnesses 4

1.3 The Four-Pillar Competency Map, Wire-Trace Anatomy & Surface Mastery (agy & claude) 5

2 Phase 1 (Modules 3 & 4): The RePPIT Daily Workflow, Context Window Physics & The 40% "Dumb Zone" **9**

2.1 Directing the Daily Workflow: Research -> Propose -> Plan -> Implement -> Verify (RePPIT) 9

2.2 Context Window Physics, The 40% "Dumb Zone" & Frequent Intentional Compaction (RPI) 10

3 Phase 2 (Module 5): Harness Engineering Layer 1 — The Three Context Primitives (`CLAUDE.md` / `GEMINI.md`), Rules & Auto-Memory **13**

3.1 The Harness Equation (Agent = Model + Harness) & The 2×2 Cybernetic Control Matrix 13

3.2 The Three Context Primitives, The 150-Instruction Cliff & The Minimalist 60-Line Map Rule 14

4 Phase 2 (Module 6): Harness Engineering Layer 2 — Progressive-Disclosure Skills, `context-hub` Grounding & Meta-MCP "Code Mode" **17**

4.1 Progressive-Disclosure Skills (`agentskills.io`) & Curated Self-Annotating Grounding (`context-hub`) 17

4.2 Ergonomic MCP Tool Design & Meta-MCP "Code Mode" (> 90% Token Reduction) 17

4.3 Empirical Skill Science (`SkillsBench`), `eval-viewer` & Trigger Calibration 18

5 Phase 2 (Modules 7 & 8): Subagents as Context Firewalls, LLM Councils, 4-Tier Governance, `Autoresearch` Ratchets & Program Design **21**

5.1 Module 7: Subagents as Context Firewalls, Multi-Agent PR Triage, `llm-council` & Agentic VCS (`git worktree` vs. Jujutsu `jj`) 21

5.2 Module 8: 4-Tier Governance, Remediation Linters, `Autoresearch` Ratchets & Program Design 22

5.3 References & Further Reading 26

1 Phase 1 (Modules 1 & 2): Natural Language to Code, The 2022–2026 Timeline & The Agentic Loop (agy & claude 101)

1.1 Why Natural Language Correlates with Code, Language Philosophy & Software Engineering Fundamentals

Source Code as Machine Instructions and an Externalized Model of Domain Understanding. Why does software engineering rigor become *more* essential—rather than obsolete—as large language models make generating syntax virtually free? Because source code serves two distinct purposes simultaneously: it is a deterministic sequence of instructions executed by a CPU, and it is an externalized **model of understanding** that encodes domain vocabulary, bounded contexts, and system invariants [1]. When engineers passively accept unreviewed, "vibe-coded" diffs without maintaining that underlying conceptual model, a repository rapidly accumulates **Cognitive Debt**: synonym drift across modules (inventing `client_id`, `cust_acct`, and `user_id` for the same customer entity), IEEE-754 floating-point currency rounding bugs in place of integer minor units (`MoneyCents`), and plausible-looking abstractions that silently violate domain invariants.

Two Philosophies of Language: Formal Picture Theory vs. Language-as-Action. How can conversational natural language steer a deterministic compiler and trigger reliable software actions? The relationship between human intent and executable code rests on two complementary theories of language originally formulated by Ludwig Wittgenstein [2, 4]:

1. **Formal Picture Theory (*Tractatus Logico-Philosophicus*, 1921):** Language is modeled as a strict, formal, 1-to-1 logical picture of reality. In computing, this corresponds directly to **programming languages, static type checkers, compilers, and deterministic test suites**, where every expression is either formally valid or rejected.
2. **Meaning as Use and Language-as-Action (*Philosophical Investigations*, 1953):** Natural language is not a static logical calculus, but an active **collaborative toolbox used between coworkers to coordinate actions** (such as a builder and an assistant calling out *"Block!"*, *"Pillar!"*, *"Slab!"*). Words acquire rigorous meaning from their rules of use inside a shared engineering practice [2, 4].
3. **Empirical Laws of Human–Agent Coding Interaction:** Controlled interactive studies of the builder–assistant paradigm across 100 human participants instructing an AI agent via natural language (`SHRDLURN` [3]) prove that two human habits mathematically predict task completion: **(a) Avoiding Synonyms** (enforcing a consistent ubiquitous domain vocabulary) and **(b) Compositionality** (packaging low-level steps into reusable, named operational abstractions—the exact foundation of agent **Skills**). When shared context is incomplete, an epistemically honest agent pauses to ask clarifying questions rather than guessing defaults.

Why Software Engineering Fundamentals Govern Agentic Coding. The primary failure mode of naive Vibe Coding occurs when a developer prompts an agent without **Software Engineering Fundamentals** [5]. Because a language model optimizes for immediate surface plausibility, an unsteered prompt cannot detect when the generated code makes destructive architectural trade-offs across **latency, availability, data consistency, security, reliability, maintainability, simplicity, and cost**. To build production software, the human engineer must act as the **Conductor** (*Orchestration over Automation*), supplying explicit non-functional constraints (p99 latency bounds, idempotency keys, ACID transaction isolation, and bounded retry budgets) and governing the agent through a progressive **Ladder of Earned Autonomy** (`openworker` [17]).

Vibe Coding, Software 3.0, and the LLM Operating System. The term **"Vibe Coding"**—coined by Andrej Karpathy in February 2025 while building rapid weekend prototypes such as *MenuGen* and `llm-council` by "fully giving in to the vibes, embracing exponentials, and forgetting that the code even exists" [6, 18]—captures the transition to **Software 3.0**. While Software 1.0 consists of human-written

explicit code and Software 2.0 consists of trained neural network weights, Software 3.0 treats natural language prompts as a programmable layer atop the **LLM Operating System** (where the *Context Window = RAM*, *Tool/MCP Calls = System Calls*, and *Disk Artifacts = Persistent Storage*). Moving from a weekend prototype to production software requires graduating from unsteered prompting to disciplined **Harness Engineering** across Phase 1 and Phase 2 of this textbook.

1.2 The 2021–2026 Architectural Evolution, The 17-Day Flagship Cadence & Model-Agnostic Harnesses

How Human–AI Software Engineering Evolved: From Manual Coding to Closing the Loop. Between 2021 and 2026, the relationship between software engineers, computing tools, and frontier AI models progressed through five distinct operational stages (Figure 1) [7]. In the first stage (**2021–2023**), engineers wrote code and documentation manually on laptops, assisted only by line-level `Tab` autocomplete. Once those first frontier models were trained, they fed directly back into the developer toolchain: first as conversational **Chatbots & Tool Protocols (2023–2024)**, where engineers generated snippets and standardized JSON tool invocation via Function Calling and the Model Context Protocol (MCP); next as repository-aware **Coding Agents (2025)**, where reinforcement learning with verifiable rewards (RLVR) enabled CLI and IDE agents (`claude` and `agy`) to read codebases, edit multi-file vertical slices, and execute test suites directly; and today (**2026**) as **Autonomous Agents & Harnesses**, where engineers orchestrate parallel subagent swarms inside governed harnesses (`git worktree`, colocated `jj`, `Temporal` durable replay, and `Omnigent` meta-harnesses) across 12–16 hour task horizons. At the frontier (**Closing the Loop**), bounded verification harnesses (`autoresearch` and kernel optimization loops) run overnight experiments to accelerate the next generation of models.

Figure 1.1 Blueprint: How Human–AI Software Engineering Evolved (2021–Now)

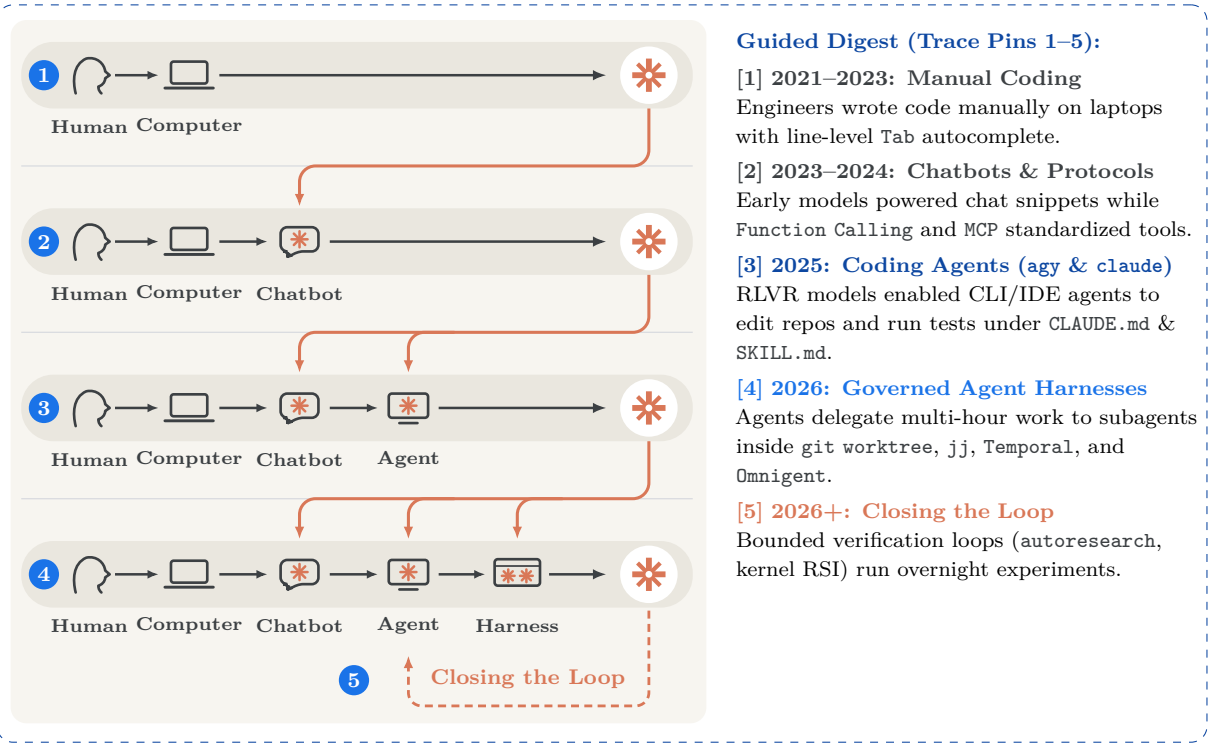


Figure 1: Visualizing how each generation of frontier AI models feeds back into the developer tools and harnesses that build the next generation, from manual coding (2021–2023) to chatbots, coding agents, governed harnesses, and recursive verification loops [7].

Why Harnesses Must Decouple from Model Weights: The 17-Day Flagship Cadence. Why is it

so dangerous to hardcode brittle, model-specific prompt workarounds into an application? Empirical release telemetry across **143 notable AI model releases (57 flagships) from 8 global laboratory groups** [8] demonstrates that the mean industry interval between flagship frontier models compressed by more than 2.1× between 2023 and 2026, dropping from **37 days in 2023** to **every 17 days in 2026** (peaking at **22 notable releases per quarter**, up from 4 per quarter in 2023). Within individual labs, the median gap between flagships fell from **124 days to 39 days at Anthropic** and from **91 days to 57 days at OpenAI**. Because a new frontier model ships roughly every two and a half weeks while autonomous task horizons double every four months on the METR benchmark to **12–16 hours** [7, 8], durable engineering teams decouple their **model-agnostic harnesses** (`CLAUDE.md/GEMINI.md`, `SKILL.md`, `aisuite`, `Omnigent`) from model weights and re-benchmark on each upgrade to prune obsolete scaffolding (Figure 2, Table 1):

Figure 1.2 Blueprint: 2022–2026 Milestones & SearchIntel Release Pace Coordinate Charts

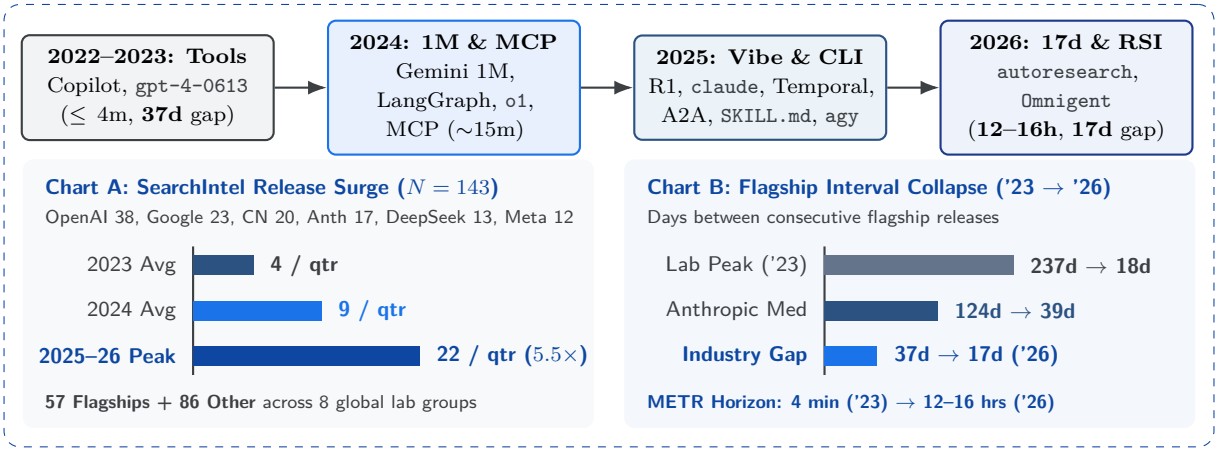


Figure 2: The verified 6-Era Architectural Timeline (2022–2026) paired with coordinate bar charts of the 8-lab model release surge (`4/qtr` → `22/qtr`) and inter-flagship gap compression (`237d` peak → `18d`; industry `37d` → `17d`) [7, 8].

Common Misconception vs. Engineering Reality: Why Prompt Tricks Decay Every 17 Days While Harnesses Compound

Common Misconception: *"AI engineering is mainly about discovering clever prompt phrasings and model-specific workarounds."* **Engineering Reality:** Because a new flagship frontier model ships every **17 days** across global labs [8], prompt workarounds tuned to one model snapshot become obsolete within weeks. Durable engineering value lives in a **model-agnostic harness**—concise `CLAUDE.md/GEMINI.md` maps, modular `SKILL.md` packages, Swiss-Army-Knife tools, and deterministic tests—that you re-benchmark on each model upgrade to prune unnecessary scaffolding.

1.3 The Four-Pillar Competency Map, Wire-Trace Anatomy & Surface Mastery (agy & claude)

The Four-Pillar AI Engineering Competency Map and the Three-Layer Agent Stack. Modern AI-enabled software engineering spans **4 Pillars and 20 Core Competencies** [5]: (1) **Building & Deploying AI Applications** (LLM foundations, grounding, agent harnesses, evaluation-driven development), (2) **Software Engineering Fundamentals** (full-stack systems, data management, system architecture, shift-left security, production operations), (3) **Using Coding Agents** across the **3-Stage Coding Agent Workflow (`01 Planning -> 02 Execution -> 03 Deployment & Monitoring`)** (directing workflows, enabling autonomy, reviewing work with browser screenshots and evaluations, customizing `CLAUDE.md/GEMINI.md`/skills/MCP/hooks, and understanding coding agent foundations), and (4) **Shaping**

Era & Stage	Model, API & Protocol Breakthroughs	Human Interaction Surface	Empirical Capability & Cadence
Late 2022 (Stage 1 → 2)	GitHub Copilot GA (Jun 2022), ReAct reasoning-and-acting loop (Oct 2022), ChatGPT (Nov 30, 2022).	Ghost-text `Tab` completion & copy-pasting code snippets from browser chat.	Single-function autocomplete ($\leq$ 1 min horizon).
2023 (Stage 2 Chatbots)	Cursor launch (Mar 2023), OpenAI **Function Calling** (`gpt-4-0613`, Jun 13), SWE-bench (Oct), **Parallel `tools` API** (Nov 6).	In-editor `@Codebase` RAG & inline `Cmd+K` single-file diff generation.	Schema-validated JSON `tool_calls` ($\approx$ 2–4 min; **37d** flagship interval, 7 Jan–Sept flagships).
2024 (Long Context, Checkpointers & MCP)	**LangGraph** checkpointers (Jan), Gemini 1.5 Pro **1M–2M context** (Feb), SWE-agent ACI (Apr), `.cursorrules` (Jun), **OpenAI `o1`** RL (Sep 12), **Anthropic MCP** (Nov 25).	Multi-file IDE Composer, stateful graph checkpointers, & static repo rule files (`.cursorrules`).	Test-time RL compute + universal MCP "USB-C" tool bus ($\approx$ 15 min; 8 Jan–Sept flagships).
Early–Mid 2025 (Stage 3 Agents & Durable Workflows)	**DeepSeek-R1 RLVR** (Jan 20), **Vibe Coding coined** (Feb 2), **Claude Code + `CLAUDE.md`** (Feb 24), **Temporal + Agents SDK / `mcp-agent`** (Mar), **Google A2A** (Apr 9), **Claude 4** (May 22), **Gemini CLI (`GEMINI.md`) + `AGENTS.md`** (Jun 25).	Terminal-native agent loops (`claude`, `gemini`) + durable workflow replay (`Temporal`); MCP (tools) + A2A (agents).	Multi-step bash/test/repair & crash-recoverable loops ($\approx$ 45–90 min; 26% pass rate; 12 Jan–Sept flagships).
Late 2025 (Skills & Antigravity)	Claude Code **Subagents, Hooks & Plugins** (Summer–Oct 9), **Agent Skills (`SKILL.md`)** (Oct 16), **Google Antigravity & Gemini 3** (Nov 18), `agentskills.io` **Standard** (Dec 18).	Agent-first Mission Control (`agy`), Plan/Walkthrough Artifacts, Browser Subagent, 3-level `SKILL.md`.	Context-isolated subagents & progressive disclosure ($\approx$ 2–4 hr horizon).
2026 – NOW (Stage 4–5 Harnesses, 17d Pace, Meta-Harnesses & RSI)	**Harness Engineering** (Feb 2026), `autoresearch` (Mar), **Durable Harnesses (`Temporal`, `Restate`, `DBOS`, `Inngest`)**, **Databricks `Omnigent` Meta-Harness** (Jun 13), **Antigravity 2.0 (`agy`)**, **Skills Map, `OpenWorker` & `chub`**, **CS146S `RePPIT` & Meta-MCP**, **17d Release Pace** (Sept 23), **Anthropic RSI** (Aug–Oct).	Designing model-agnostic 4-tier harnesses, cross-harness meta-harnesses (`Omnigent`), durable workflows, & (`model`, `date`)-pinned eval ratchets.	**17d flagship interval** (143 releases / 57 flagships; Anthropic **124d → 39d**, OpenAI **91d → 57d**), **12–16 hr METR horizon**, > 80% AI code, **52× kernel RSI**.

Table 1: The 2022–2026 Architectural & Human-Interaction Timeline paired with the 17-day flagship release cadence and 5-Stage Evolution [7, 8].

the Build (rapid Idea → MVP → Production small-batch loops and high-agency ownership). Architecturally, these competencies map onto the **3-Layer Agent Stack** (aisuite and openworker [17]) illustrated in Figure 3 below: **Layer 1** (Chat Completions API) → **Layer 2** (Agents API / Toolkits / MCP) → **Layer 3** (Governed Agent Harness).

Wire-Trace Anatomy of the Agentic Loop and the "Swiss Army Knife" Tool Principle. At its core, a minimal coding agent is a 200-line while loop with four primitive tools (read, write, edit, bash) that repeatedly sends the conversation array to an LLM until the model emits no further tool_calls [9]. Inspecting a production coding agent through an HTTP/JSONL proxy wire trace reveals why naive Model Context Protocol (MCP) configurations degrade performance: loading 110–150 granular CRUD tools (such as one tool per REST endpoint) injects **10,000+ lines of JSONL schema definitions** for a 600-line coding session, starving the context window and confusing tool selection. Production harnesses therefore enforce the ***"Swiss Army Knife" Tool Principle** [9]: pair a lean system prompt (≈ 50–80 lines) with a compact set of general-purpose, composable primitives (Bash, Read, Edit, Grep, Glob, LSP, SQL) that leverage the model's post-trained CLI and Git fluency.

Figure 1.3 Blueprint: 4-Pillar Skills Map, 3-Layer Stack & Wire-Trace Token Chart

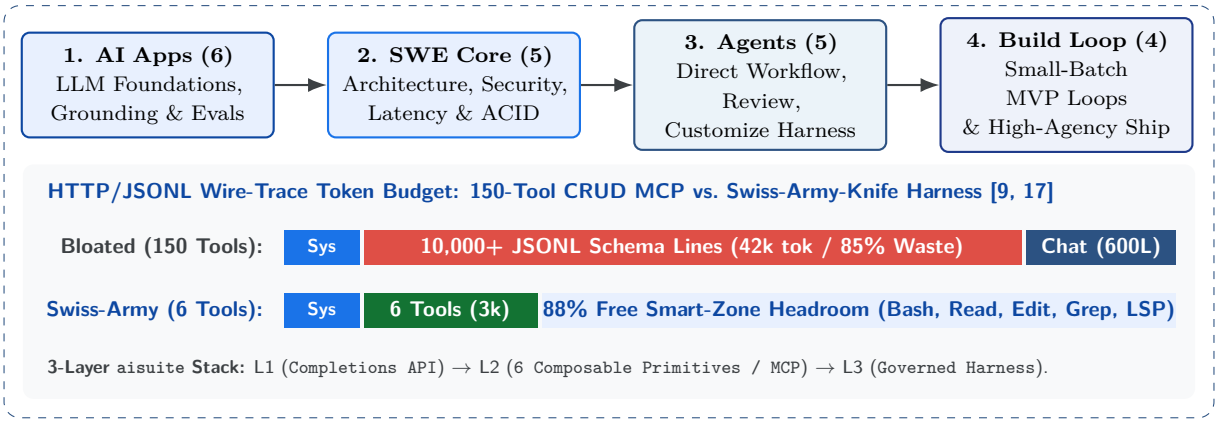


Figure 3: The 4-Pillar AI Engineering Skills Map [5], the aisuite 3-Layer Agent Stack [17], and HTTP/JSONL wire-trace token breakdown contrasting Swiss-Army-Knife primitives against 150-tool schema bloat [9].

Surface Mastery Across Google Antigravity (agy) and Anthropic Claude Code (claude). Both Google Antigravity (agy) [10] and Anthropic Claude Code (claude) [11] implement the **Core Agentic Loop (Gather Context -> Take Action -> Verify Results)** and the **3-Stage In-Editor Interaction Cycle** (Stage 1: Implicit IDE state vs. Explicit @file pinning → Stage 2: Plan vs. Execute, queued messages, real-time tool monitoring → Stage 3: Interactive diff acceptance):

Dual-Branch Quickstart (agy & claude 101) & The Ladder of Earned Autonomy (Labs 01–02)

```
# 1. Clone & run the <1s stdlib self-diagnosis across all 8 progressive engineering labs
$ ./self_diagnose_all.sh
# [ALL PASSED] 8/8 Hands-On Vibe-Coding-Course Labs Verified!

# 2. Google Antigravity (agy branch): Mission Control, IDE (Cmd+L, Tab-to-Jump) & CLI
$ agy models                                          # List frontier & fast models
$ agy -p "Audit latency & retry bounds" --model gemini-3.1-pro
# ~/.gemini/antigravity-cli/settings.json permission modes:
#   "request-review" -> "proceed-in-sandbox" -> "always-proceed" -> "strict"
# Commands: /plan, /artifact, /grill-me, /browser, /goal, /schedule, /config, !
```

```
# 3. Anthropic Claude Code (claude branch): Terminal, IDE & Desktop/Web
$ claude -p "Trace tool schema token overhead" --model claude-opus-4-7
# Shift+Tab cycles permission modes (.claude/settings.json):
#   Manual ("default") -> Auto-Accept ("acceptEdits") -> Plan ("plan") -> Auto ("auto")
# Commands: @file, ! (Bash), /context, /compact, /clear, /rewind (Esc+Esc), /btw
```

System Design Invariant & Tie-Breaker

Student Lab Guide for Chapter 1: Completing Labs 01 & 02 (TaskFlow Platform)

- **Lab 01 (labs/lab_01_vibe_coding_and_earned_autonomy) — Architecture-Guided Steering & Earned Autonomy:** Inspect starter/vibe_prompt_and_autonomy_blob.py and implement work/prompt_steerer.py (REQ-0101..0106, test_req0101..0106) to enforce canonical AccountId/MoneyCents value objects, 5-dimensional engineering steering, 4-tier agy/claude permission mapping, hard-floor safety interlocks, and provenance audit logs. Verify with ./labs/lab_01_vibe_coding_and_earned_autonomy/self_diagnose.sh.
- **Lab 02 (labs/lab_02_agent_loop_and_harness_wire_trace) — The Core Agentic Loop & Wire-Trace Inspector:** Inspect starter/bloated_wire_agent.py and implement work/wire_trace_inspector.py (REQ-0201..0206, test_req0201..0206) to profile HTTP/JSONL wire tokens across system, CLAUDE.md/GEMINI.md, tool schemas, and messages; verify why consolidating 150 granular CRUD tools into Swiss-Army-Knife primitives cuts startup schema overhead by $> 85\%$; and run ./labs/lab_02_agent_loop_and_harness_wire_trace/self_diagnose.sh.

2 Phase 1 (Modules 3 & 4): The RePPIT Daily Workflow, Context Window Physics & The 40% "Dumb Zone"

2.1 Directing the Daily Workflow: Research -> Propose -> Plan -> Implement -> Verify (RePPIT)

Matching Workflow Ceremony to Task Complexity and Blast Radius. Effective engineering with coding agents requires calibrating workflow ceremony to the blast radius of a change [5, 9]. Applying a zero-shot prompt to a multi-module stateful feature fails just as reliably as running a five-stage architectural review for a single-line typo. Production teams classify daily work across four complexity tiers:

- **Tier 1 — Copy, Typo, or Single-Line Tweak:** Direct prompt with immediate inline diff review.
- **Tier 2 — Small Bounded Multi-File Change:** Single-pass Plan Mode (`/plan` in `agy` or `Shift+Tab` Plan Mode in `claude`) prior to editing files.
- **Tier 3 — Medium Feature or Stateful Endpoint:** The complete five-step `RePPIT` **Workflow (`Research -> Propose -> Plan -> Implement -> Test/Verify`)** with a mandatory post-proposal context reset [9].
- **Tier 4 — Large Cross-Cutting Refactor or Migration:** Multi-session stacked vertical slices orchestrated with isolated read-only exploration and review subagents.

Defeating "Silent Assumption Syndrome" via Socratic Inversion of Control (`/grill-me` & Plan Mode). When handed an underspecified feature request, an unsteered coding agent exhibits **Silent Assumption Syndrome**: rather than pausing to ask questions, it silently invents database schemas, authentication policies, and pagination defaults to complete the turn. Both Google Antigravity (via `/grill-me` [10]) and Claude Code (via Plan Mode questioning [11]) enforce **Inversion of Control (IoC)**: the engineer instructs the agent to act as a Socratic interviewer that surfaces edge cases, failure modes, and missing requirements *before* drafting a single line of architecture or implementation code.

The Five-Step `RePPIT` Workflow, Orthogonal Proposals & Post-Proposal Context Reset. Synthesizing structured daily agent workflows [9, 11] with iterative self-reflection [17], the five-step `RePPIT` loop illustrated in Figure 4 governs feature development from discovery to verified commit:

1. **1. Research (`research_codebase`):** Read-only discovery (`@file`, `Grep`, `Glob`) that strictly documents *what exists today* with exact `file:line` citations—explicitly forbidden from proposing solutions—to build both the agent's grounded context and the **human engineer's mental model**.
2. **2. Propose (Two Orthogonal Proposals + Post-Proposal Context Reset):** Instruct the agent to generate **two genuinely orthogonal architectural proposals** (for example, Proposal A: synchronous in-memory token bucket vs. Proposal B: distributed SQLite/Redis sliding window) with explicit trade-off matrices. **Post-Proposal Context Hygiene Rule:** *The moment you select Proposal B, immediately persist Proposal B to disk and clear the context window (`/clear`) so the rejected Proposal A tokens do not remain in attention and contaminate implementation [9].*
3. **3. Plan (`Implementation Plan` Artifact + `Out of Scope` Guardrails):** Generate a file-and-line-targeted implementation plan (Antigravity's `Implementation Plan` Artifact with inline review comments and **Proceed** gate, or Claude Code Plan Mode via `Ctrl+G`) that includes an explicit `Out of Scope / Do NOT Touch` section to block agent scope creep.
4. **4. Implement with Phase-Based Model Routing:** Use a **frontier reasoning model** (`gemini-3.1-pro` or `claude-opus-4-7`) for *Research, Propose, and Plan*, then route *Implement* to a **fast execution model** (`gemini-3.1-flash` or `claude-sonnet-4-6`) once all architectural ambiguity is eliminated.

5. **5. Verify, Reflect & Commit (Self-Reflection + /browser Run Receipts):** Run deterministic tests, apply the three-step **Reflection loop (`Generate -> Reflect -> Refine`** [17]) where the agent audits its own diff against domain invariants, verify live UI/API behavior via Antigravity's `/browser` Subagent + `Walkthrough` Run Receipt Artifact (or Playwright MCP screenshots), and use time-travel recovery (`Undo changes up to this point` in `agy` or `/rewind` [Esc+Esc] in `claude`) if a branch diverges.

Figure 2.1 Blueprint: The 5-Step RePPIT Workflow & Post-Proposal Context Reset

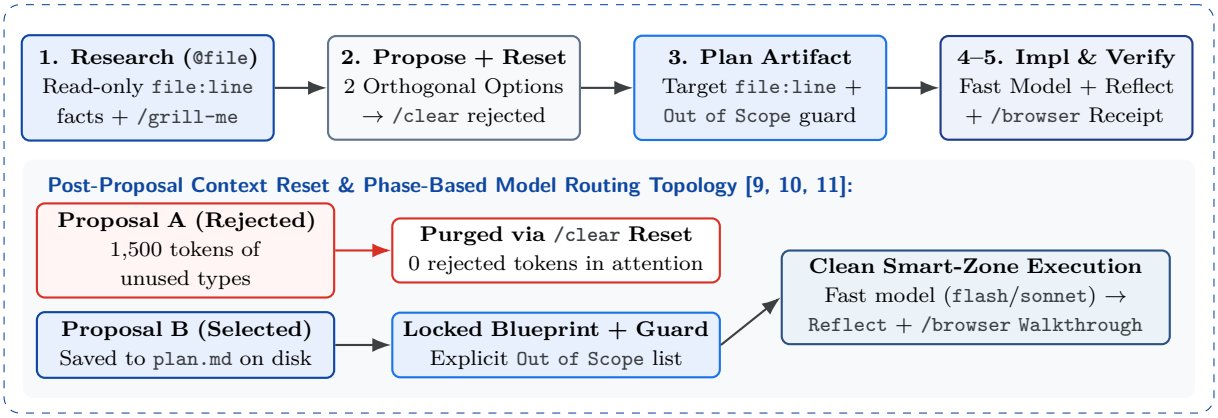


Figure 4: Five-step Phase 1 daily workflow (`Research -> Propose -> [Context Reset] -> Plan -> Implement -> Verify`) across Google Antigravity (`agy`) and Claude Code (`claude`) [9, 10, 11].

2.2 Context Window Physics, The 40% "Dumb Zone" & Frequent Intentional Compaction (RPI)

Stateless Transformer Physics and the 40% "Dumb Zone" Phase Transition. The first law of Context Engineering states that a language model is a stateless function of its current token array: NextAction = LLM(ContextWindow) [12]. Because transformer self-attention computes pairwise interactions across every token in the window, *not all 1M tokens are created equal.* As plotted in Figure 5, agent reasoning exhibits a sharp empirical phase transition at **40% context utilization**:

- **The "Smart Zone" (0%–40% Context Utilization):** System instructions, `CLAUDE.md/GEMINI.md` invariants, and pinned `file:line` snippets receive high attention weight; tool calls remain surgical and accurate.
- **The "Dumb Zone" (> 40% Context Utilization / Context Rot):** Once raw `grep` dumps, 2,000-line stack traces, and failed debugging turns push utilization past 40%, attention diffuses (*Context Rot*). The agent forgets root constraints, hallucinates function signatures, and enters repetitive error loops [11, 12].

Mechanical Defenses Against Context Rot: Dual Repository Views and Output Redirection. Two mechanical practices prevent verbose repository files and command logs from flooding the context window [18]:

1. **Dual Human vs. LLM Repository Views (`rendergit`):** Flattening a Git repository into a **Human View** (syntax-highlighted HTML with a navigable tree sidebar) alongside a compact **LLM View** (structured CXML-tagged text stripped of binary and lockfile noise) maximizes signal-to-noise during codebase ingestion.
2. **Context Backpressure via Output Redirection (`autoresearch program.md`):** Never allow verbose build, test, or training logs to stream directly into the agent's stdout tool response. Redirect all command output to disk via `uv run train.py > run.log 2>&1`, then extract only the summary metric lines via `grep "^val_bpb:\|^peak_vram_mb:" run.log` (or `tail -n 50 run.log` on crash).

Frequent Intentional Compaction (FIC), The RPI Loop, and the Human Leverage Pyramid. Rather than waiting for automatic compaction at 95% capacity (after the agent has already spent 20 turns inside the Dumb Zone), engineers practice **Frequent Intentional Compaction (FIC)** via the RPI **Loop (Research -> Plan -> Implement)** [12]: each phase writes a verified, line-referenced artifact to disk (research.md → plan.md) and immediately runs /clear (or /compact [focus] / /compress) so the next phase starts fresh at < 15% utilization. For tangential questions during coding, Claude Code's /btw runs a sandboxed side-query whose tokens are automatically pruned from the conversation history. By the **Human Leverage Pyramid** (Figure 5), 1 bad line in research.md corrupts 10 lines of plan.md and generates **1,000 lines of broken code**—making human review of research.md and plan.md the highest-leverage activity in software engineering [12].

Figure 2.2 Blueprint: 40% Dumb-Zone Physics Curve, Sawtooth RPI & Tonal Leverage Pyramid

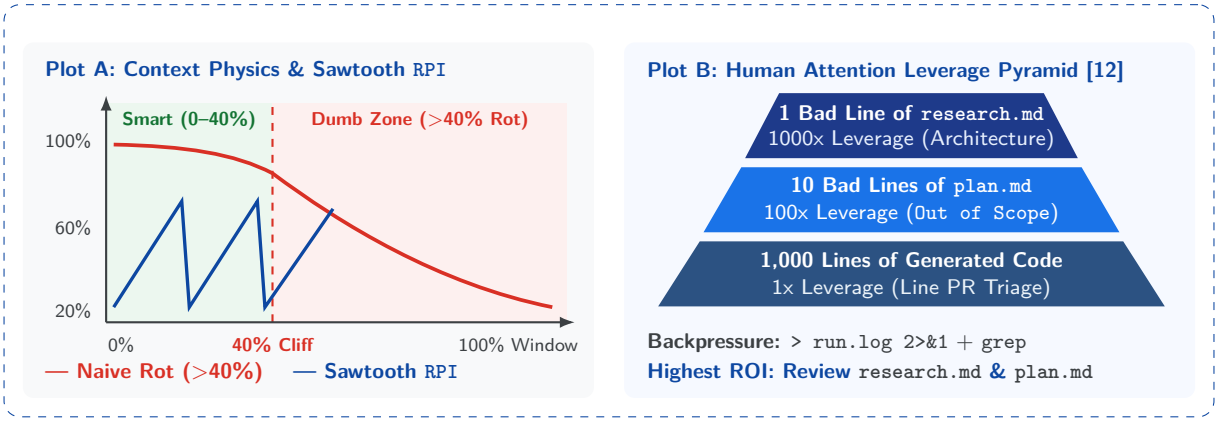


Figure 5: Context Window Physics: keeping utilization inside the 0%–40% Smart Zone via output-redirection backpressure, Sawtooth RPI Intentional Compaction, and the Tonal Human Leverage Pyramid [12, 18].

Stop and Think: Why Do Agents Hallucinate More at Turn 25 of a 1M-Token Context Window?

Question: If your model supports a 1,000,000-token context window, why shouldn't you leave every raw grep dump, rejected proposal, and 5,000-line test log in chat until automatic compaction triggers at 95%? **Answer:** Because transformer self-attention computes pairwise weights across *all* tokens in ContextWindow [12]. Once noisy tool outputs and failed turns push utilization past **40% (the Dumb Zone)**, attention over your standing CLAUDE.md/GEMINI.md rules and type constraints dilutes, causing repetitive error loops. Always redirect logs (> run.log 2>&1) and compact intentionally (RPI + /clear).

System Design Invariant & Tie-Breaker

Student Lab Guide for Chapter 2: Completing Labs 03 & 04 (TaskFlow Platform)

- **Lab 03 (labs/lab_03_reppit_workflow_and_reflection) — End-to-End RePPIT Workflow & Reflection:** Inspect starter/impulsive_feature_coder.py and implement work/reppit_orchestrator.py (REQ-0301..0306, test_req0301..0306): enforce Socratic requirement clarification (/grill-me / Plan Mode), generate 2 orthogonal architectural proposals, trigger a mandatory **post-proposal context reset**, emit a line-targeted plan with explicit Out of Scope / Do NOT Touch guardrails, apply phase-based model routing, and validate with ./labs/lab_03_reppit_workflow_and_reflection/self_diagnose.sh.
- **Lab 04 (labs/lab_04_context_physics_and_rpi_compaction) — Context Budget Profiler & RPI Compaction Engine:** Inspect starter/dumb_zone_session_accumulator.py

Workflow / Physics Step	**Google Antigravity (`agy` Branch)**	**Anthropic Claude Code (`claude` Branch)**	**Context & Engineering Invariant**
Requirement Interview	`/grill-me` Socratic interviewer before planning.	Plan Mode (`Shift+Tab`) clarifying questions prompt.	Defeats Silent Assumption Syndrome before architecture.
Orthogonal Proposals	Generate Proposal A vs. B in Mission Control; `/clear` after choice.	Generate 2 orthogonal designs; save winner & run `/clear`.	Eliminates rejected-proposal token pollution in attention.
Plan & Scope Guard	`/plan` → `Implementation Plan` Artifact + **Proceed** button.	Plan Mode (`Ctrl+G`) → `plan.md` with `Out of Scope` block.	Reviewing 1 line of plan saves 100 lines of broken code.
Context Audit & FIC	`/stats` (utilization), `/compress`, `/clear`, `> run.log 2>&1`.	`/context` (token breakdown), `/compact [focus]`, `/clear`, `/btw`.	Never code past the **40% Dumb Zone** threshold.
Verification & Undo	`/browser` Subagent + `Walkthrough` Receipt; `Undo changes`.	Playwright MCP + `translation-agent` reflection; `/rewind`.	Behavioral + functional proof before git commit.

Table 2: Phase 1 daily workflow (`RePPIT`) and context-window hygiene controls across `agy` and `claude`.

and implement `work/context_compactor.py` (`REQ-0401..0406`, `test_req0401..0406`) to flag any turn crossing the **40% Dumb Zone threshold**, rewrite verbose shell invocations with output-redirection backpressure (`> run.log 2>&1`), and execute **Frequent Intentional Compaction** into a line-cited `research.md` + `plan.md` bundle (`./labs/lab_04_context_physics_and_rpi_compaction/self_diagnose.sh`).

3 Phase 2 (Module 5): Harness Engineering Layer 1 — The Three Context Primitives (CLAUDE.md / GEMINI.md), Rules & Auto-Memory

3.1 The Harness Equation (Agent = Model + Harness) & The 2×2 Cybernetic Control Matrix

The Cybernetic Harness Equation (Agent = Model + Harness) and Control Matrix. Phase 2 transitions from interactive single-session prompting to **Harness Engineering**: designing the surrounding execution environment, standing instructions, tools, and feedback sensors that transform a stochastic foundation model into a reliable software engineering system [13, 14]. Formally, Agent = Model + Harness. While human engineers retain ownership of the outer **Why Loop** (domain value, product intent, and architectural invariants) and delegate the nested **How Loops** to coding agents, every component of an agent harness falls into the **2×2 Cybernetic Control Matrix** [13] illustrated in Figure 6:

1. **Guides (Feedforward Control — Steering Before Action) × Inferential (LLM):** Root `CLAUDE.md` / `GEMINI.md` (`AGENTS.md`), path-scoped `.claude/rules/` & `.agents/rules/`, and on-demand `SKILL.md` manuals.
2. **Guides (Feedforward Control — Steering Before Action) × Computational (CPU):** Static type annotations, Pydantic tool schemas, container filesystem sandboxes (`NVIDIA OpenShell`), and tool allowlists.
3. **Sensors (Feedback Control — Verification After Action) × Computational (CPU):** Deterministic unit and adversarial test suites, `PreToolUse` Exit-Code-2 gates, and remediation-aware architectural linters.
4. **Sensors (Feedback Control — Verification After Action) × Inferential (LLM):** Context-isolated `code-reviewer` subagents, self-reflection loops (`translation-agent`), and anonymized cross-model peer review (`llm-council`).

Figure 3.1 Blueprint: The 2×2 Cybernetic Harness Matrix (Guides vs. Sensors × CPU vs. LLM)

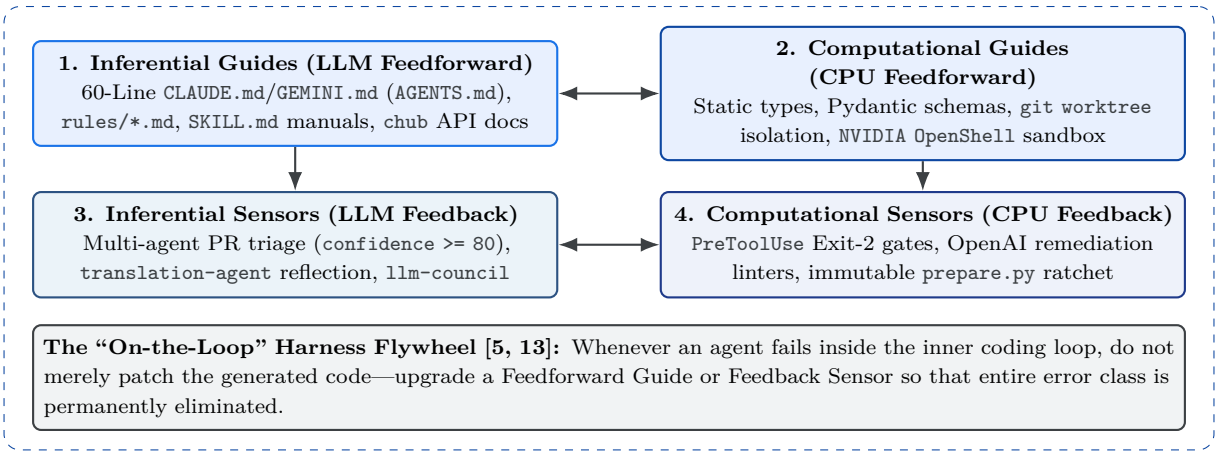


Figure 6: The 2×2 Cybernetic Control Matrix mapping Feedforward Guides and Feedback Sensors across deterministic CPU checks and semantic LLM reasoning [13].

3.2 The Three Context Primitives, The 150-Instruction Cliff & The Minimalist 60-Line Map Rule

The Three Context Primitives (Prompts, Rules, Skills) and the < 20k Standing Context Budget. To prevent startup instructions from pushing an agent toward the 40% Dumb Zone before a single repository file is read, Harness Engineering partitions instructions across three distinct primitives (Table 3): **(1) Prompts / Slash Commands** (per-turn ephemeral task steering), **(2) Rules** (standing workspace or glob-scoped invariants in CLAUDE.md/GEMINI.md and rules/*.md), and **(3) Skills** (progressive-disclosure procedural manuals in SKILL.md). By the **Standing Context Economy Rule**, total always-on startup context (system prompt + root context file + active Level 1 skill metadata + auto-memory index) must remain strictly under **< 20k tokens** (< 5% of a 200k–1M context window).

The 150-Instruction Compliance Cliff and the Minimalist 60-Line Root Map Rule. A pervasive anti-pattern in early agent adoption is treating CLAUDE.md or GEMINI.md as a 600-line dumping ground for every style preference, database schema, and historical bug fix. Empirical instruction-following evaluations demonstrate the **150-Instruction Compliance Cliff**: language-model adherence to standing instructions remains > 92% up to ≈ 100–150 discrete rules, then degrades sharply as competing rules dilute attention and overconstrain codebase exploration [9, 14]. Production repositories therefore enforce the **Minimalist 60-Line Map Rule** illustrated in Figure 7:

- **Keep Root CLAUDE.md / GEMINI.md to ≤ 60 Lines:** Include only (a) exact build, test, lint, and server commands, (b) a concise directory and module architecture map, and (c) 3–5 non-negotiable domain invariants.
- **Symlink CLAUDE.md <-> AGENTS.md (and GEMINI.md):** Maintain a single source of truth across Google Antigravity (agy), Claude Code (claude), and multi-tool repositories.
- **Delegate Deep Documentation via @docs/*.md Lazy Imports & Path-Scoped Rules:** Reference deep architecture or testing manuals via @docs/architecture.md (0 startup tokens until pulled on demand) and scope file-specific rules using .claude/rules/*.md (paths: globs) and .agents/rules/*.md (trigger: glob | model_decision).

Figure 3.2 Blueprint: 150-Instruction Compliance Cliff Curve, 60-Line Map & Auto-Memory

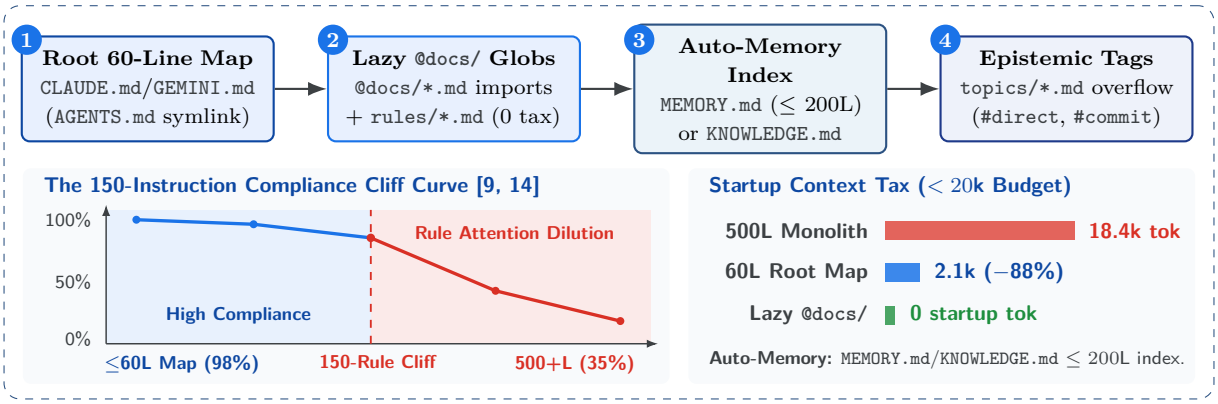


Figure 7: Defeating the 150-Instruction Compliance Cliff by pairing a ≤ 60-line root CLAUDE.md/GEMINI.md (AGENTS.md) map and lazy @docs/*.md imports with cross-session Auto-Memory (MEMORY.md / KNOWLEDGE.md) [9, 14].

Cross-Session Auto-Memory (MEMORY.md vs. KNOWLEDGE.md) and Epistemic Provenance. While CLAUDE.md and GEMINI.md are human-authored and checked into Git, **Auto-Memory** captures agent-discovered learnings, debugging patterns, and user corrections across sessions [10, 11]:

- **Claude Code Auto-Memory (claude branch):** Stored at ~/.claude/projects/<project>/memory/

MEMORY.md. Because <project> is derived from the **git repository root**, all git worktree checkouts and subdirectories share one unified memory store. Only the first **200 lines / 25 KB** of MEMORY.md are auto-loaded at startup; detailed notes live in topics/*.md and are managed via /memory.

- **Google Antigravity Knowledge Items (agy branch):** Stored at ~/.gemini/antigravity/knowledge/<project>/KNOWLEDGE.md (scope: workspace) with topic files in topics/*.md, managed via /memory show|add|refresh. Every recorded fact carries strict **epistemic provenance tags**: #direct (reserved *exclusively* for explicit user instructions, never agent inferences), #commit:<sha>, #time:<YYYY-MM-DD>, and #session:<id>.

Common Misconception vs. Engineering Reality: More Rules in CLAUDE.md / GEMINI.md Always Improve Agent Accuracy

Common Misconception: Appending every team convention, schema table, and postmortem note directly into a 600-line root CLAUDE.md or GEMINI.md makes the coding agent smarter and safer.

Engineering Reality: Beyond ≈ 150 discrete standing rules, instruction adherence drops down the **150-Instruction Compliance Cliff** (98% $\rightarrow$ 35%) while burning $>$ 18k startup tokens every turn [9, 14]. Keep the root map ≤ 60 lines and delegate domain details to lazy @docs/*.md imports and path-scoped rules/*.md globs.

Context Primitive	Google Antigravity (agy Branch)	Anthropic Claude Code (claude Branch)	Token Budget & Scope Rule
Root Map File	GEMINI.md & symlinked AGENTS.md.	CLAUDE.md & symlinked AGENTS.md (/init).	≤ 60 lines; exact build/test/lint commands + module map.
Lazy Doc Imports	@docs/architecture.md on-demand references.	@docs/architecture.md & @docs/testing.md.	0 startup tokens until pulled for relevant task.
Path-Scoped Rules	.agents/rules/*.md (trigger: glob | model_decision).	.claude/rules/*.md (paths: ["src/api/**"]).	Loaded only when matching files enter context.
Auto-Memory Store	~/.gemini/antigravity/knowledge/<proj>/KNOWLEDGE.md + #direct.	~/.claude/projects/<repo>/memory/MEMORY.md (worktree-shared).	≤ 200 lines / 25 KB index; overflow in topics/*.md.

Table 3: 1-to-1 architectural correspondence across the Three Context Primitives and Auto-Memory in agy and claude.

Copy-Pasteable Template: Minimalist 60-Line Root Map (CLAUDE.md / GEMINI.md) & Provenance Memory (Lab 05)

```
# Settlement Platform - Root Map (Symlinked: CLAUDE.md <-> AGENTS.md <-> GEMINI.md)
## 1. Exact Build, Test & Lint Commands
- Run all unit & adversarial tests (<1s): `./self_diagnose_all.sh`
- Single lab: `CI=true python3 -m unittest discover -s labs/lab_05_context/tests`
## 2. High-Level Architecture Map
- `src/domain/`: Integer value objects (`AccountId`, `MoneyCents`; NO floats).
- `src/api/`   : Idempotent handlers (`.claude/rules/api.md`, `.agents/rules/api.md`).
## 3. On-Demand Deep Documentation (0 Startup Tokens Until Pulled)
- Domain invariants & state machines : @docs/architecture.md
- Deterministic test & hook rules    : @docs/testing-conventions.md

# Cross-Session Auto-Memory Entry Format (MEMORY.md / KNOWLEDGE.md):
```

```
- Use integer minor units (MoneyCents) across payloads #direct #time:2026-10-09
- Batch worker uses SQLite WAL mode #commit:8f3a19c #session:b42e10a
```

System Design Invariant & Tie-Breaker

Student Lab Guide for Chapter 3: Completing Lab 05 (`TaskFlow` Platform)

- **Part A — Refactor Monolithic Instructions into a $\leq$ 60-Line Root Map (`labs/lab_05_context_primitives_and_memory`):** Inspect `starter/monolithic_claude_md_dump.py` and implement `work/context_compiler.py` (`REQ-0501..0506`, `test_req0501..0506`) to decompose a bloated 400-line instruction file into a concise root `CLAUDE.md` / `GEMINI.md` (`AGENTS.md` symlinked) map plus `@docs/*.md` lazy imports and path-scoped `.agents/rules/` & `.claude/rules/`, verifying < 150 total standing rules.
- **Part B — Enforce Git-Worktree Memory Sharing & `#direct` Provenance:** Complete the memory store validator so that canonical git repository roots share a single memory index across worktrees, superseded commits are pruned under the 200-line startup cap, and agent-inferred observations are blocked from carrying the `#direct` tag (`./labs/lab_05_context_primitives_and_memory/self_diagnose.sh`).

4 Phase 2 (Module 6): Harness Engineering Layer 2 — Progressive-Disclosure Skills, context-hub Grounding & Meta-MCP "Code Mode"

4.1 Progressive-Disclosure Skills (agentskills.io) & Curated Self-Annotating Grounding (context-hub)

Three-Level Progressive Disclosure in Agent Skills (agentskills.io). An Agent Skill is a modular, filesystem-backed procedural manual (`.agents/skills/<name>/SKILL.md` in `agy` and `.claude/skills/<name>/SKILL.md` in `claude`) that teaches an agent how to execute a repeatable engineering workflow. To prevent a library of skills from exhausting the $<$ 20k standing token budget at session startup, the open **Agent Skills Specification** (`agentskills.io` [15]) enforces the **3-Level Progressive Disclosure** hierarchy shown in Figure 8:

1. **Level 1 — Startup Metadata ($\approx$ 100 tokens per skill):** Only the YAML frontmatter `name` (1..64 lowercase alphanumeric/hyphen characters matching the directory name) and `description` ($\leq$ 1024 characters specifying positive `Use when...` triggers and negative `Don't use for...` boundaries) are loaded into the startup system prompt.
2. **Level 2 — Core Procedural Manual (`SKILL.md` $\leq$ 500 lines / $<$ 5,000 tokens on trigger):** Loaded into context only when invoked explicitly via `/skillname` or matched by the semantic router.
3. **Level 3 — On-Demand Scripts & Reference Manuals (`scripts/`, `references/`, `assets/` — 0 startup tokens):** Domain sub-guides and deterministic verification scripts are read or executed on demand, keeping unused branches out of the context window.

Versioned API Grounding and Self-Improving Session Annotations (context-hub). Even frontier models frequently hallucinate deprecated SDK methods or webhook parameters when third-party APIs evolve faster than pretraining cutoffs. The open-source `context-hub` CLI (`@aisuite/chub`) and companion `get-api-docs/SKILL.md` [17] pair progressive disclosure with a **self-improving session memory loop**:

- **Versioned, Language-Specific API Retrieval:** Rather than scraping noisy web pages, the agent executes `chub search` and `chub get openai/chat -lang py` to retrieve curated, version-pinned Markdown documentation.
- **Self-Annotating Learning Loop (`chub annotate`):** When an agent discovers a non-obvious API quirk during a session, it persists a local annotation via `chub annotate stripe/api "Needs raw body for webhook verification"`. Subsequent sessions fetching `stripe/api -with-annotations` automatically receive the verified note (sandboxed as untrusted input by default) and can report documentation quality upstream via `chub feedback up|down`.

4.2 Ergonomic MCP Tool Design & Meta-MCP "Code Mode" ($>$ 90% Token Reduction)

Why Naive 1-to-1 REST-to-MCP CRUD Wrappers Degrade Agent Reasoning. Auto-generating one Model Context Protocol (MCP) tool per REST endpoint cripples coding agents in two ways [9]. Traditional REST APIs expose low-level CRUD primitives designed for high-frequency CPU code; when an LLM must invoke four sequential CRUD tools to answer a single question, intermediate JSON payloads flood the context window and push the session past the 40% Dumb Zone. Simultaneously, loading 110–150 fine-grained tool schemas injects over 10,000 lines of JSONL schema definitions at session startup.

The Four Laws of Ergonomic Agent Tools and Meta-MCP "Code Mode" (search + execute). Production MCP servers obey four ergonomic design laws [9] illustrated in Figure 8:

1. **Law 1 — Design Around Complete User Outcomes, Not Granular CRUD:** Consolidate multi-step lookup chains into a single outcome tool (such as `resolve_and_verify_settlement`) or unify fragmented search endpoints (`search_meetings`, `search_transcripts`, `search_slack`) into `search_resource(resource_type=...)`.
2. **Law 2 — Enforce Strict Typed Schemas (Pydantic / TypeScript) Over **kwargs:** Replace untyped `dict` payloads with strict schema constraints, enumerations, and bounded pagination defaults.
3. **Law 3 — Return Actionable, Self-Repairing Typed Error Exceptions:** Format tool error strings with explicit recovery instructions so the agent repairs invalid arguments on the very next turn.
4. **Law 4 — Meta-MCP "Code Mode" (search + execute Meta-Tools for > 90% Token Savings):** Instead of injecting 150 tool schemas (> 10,000 lines of JSONL) at startup, expose *only two meta-tools*: `search(query)` (which dynamically queries the tool registry on demand and returns only the 2–3 matching schemas) and `execute(code)` (which executes a generated script chaining those calls inside an isolated sandbox VM and returns only the distilled result), cutting MCP schema token overhead by **> 90%**.

Figure 4.1 Blueprint: 3-Level agentskills.io, chub, Meta-MCP & SkillsBench Charts

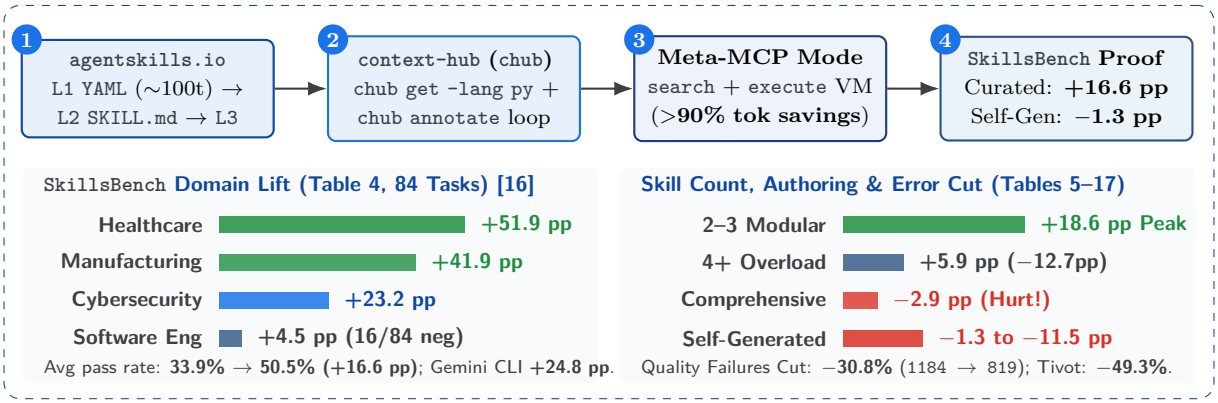


Figure 8: Layer 2 Harness Engineering combining 3-Level Progressive Disclosure (`agentskills.io` [15]), `context-hub` (`chub` [17]), Meta-MCP "Code Mode" (> 90% token savings [9]), and `SkillsBench` domain, quantity, and failure-reduction charts [16].

4.3 Empirical Skill Science (SkillsBench), eval-viewer & Trigger Calibration

Empirical Skill Benchmarks: Curated Procedural Skills (+16.6 pp) vs. Self-Generated Traps (−8.1 to −11.5 pp). Large-scale empirical evaluation across **87 tasks in 8 domains** and **9,396 trajectories** on Claude Code, Gemini CLI, and Codex CLI (`SkillsBench` [16], Table 4) quantifies when skills help and when they hurt. Human-curated skills grounded in observed failure trajectories raise average pass rates from **33.9% to 50.5% (+16.6 pp)**—including **+24.8 pp on Gemini CLI + Gemini 3.1 Pro** (`36.0% -> 60.8%`) and **+18.2 pp on Claude Code + Opus 4.7** (`43.0% -> 61.2%`). Conversely, instructing an LLM to **self-generate its own skill zero-shot before solving drops pass rates below the No-Skills baseline across every harness** (**−8.1 pp on Claude Code**, **−11.3 pp on Codex**, **−11.5 pp on Gemini CLI**) because zero-shot self-generated skills lock in hallucinated assumptions (such as multiplying unitless 3D mesh coordinates by `1000.0` in `3d-scan-calc`).

The Skill Overload Penalty, Maturity Lifecycle, and Signal Detection Theory (d', c). Empirical quantity and length ablations [16] reveal a strict **Skill Overload Penalty**: providing `2-3` focused skills achieves peak lift (**+18.6 to +19.0 pp**), whereas loading `>= 4` skills drops lift to **+5.9 to +10.1 pp** and

>10 overlapping skills cause severe router confusion; similarly, `Compact` (**+19.0 pp**) and `Standard` (**+21.5 pp**) skills dramatically outperform `Comprehensive` dumps (**−2.9 to +0.7 pp**). Engineering teams therefore govern skills through a **3-Stage Maturity Lifecycle** (Figure 9): (1) *Author from observed zero-shot failure deltas*, (2) *Benchmark via paired `with_skill` vs. `without_skill` runs in `eval-viewer` (`grading.json`, `benchmark.json`, 11/11 binomial zero-flake gate)* while calibrating trigger descriptions via Hautus (+0.5) Signal Detection Theory ($d' = \Phi^{-1}(\mathrm{HR}_{\mathrm{adj}}) - \Phi^{-1}(\mathrm{FA}_{\mathrm{adj}}) > 2.0$, criterion bias $|c| \leq 0.25$), and (3) *Prune obsolete skills when newer frontier models absorb them natively* [5, 16].

Figure 4.2 Decision Tree: `chub` vs. Meta-MCP vs. `SKILL.md` vs. `references/`

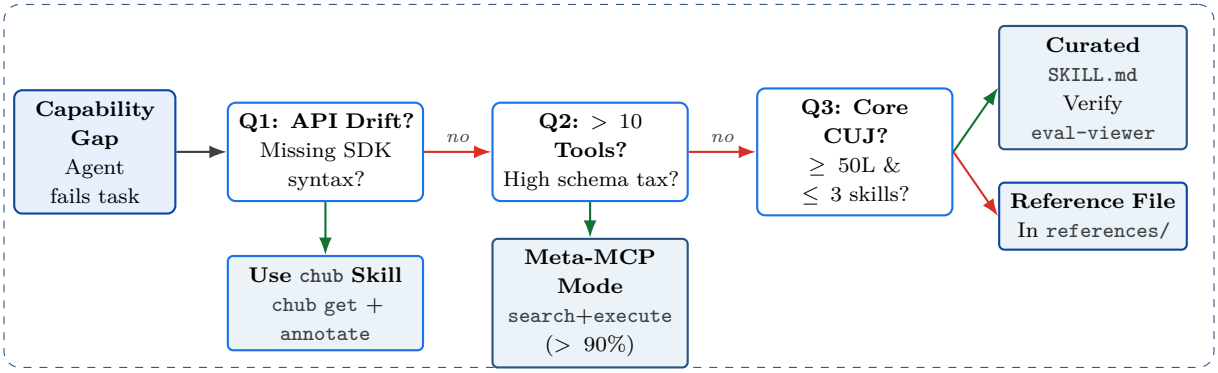


Figure 9: Left-to-Right decision tree routing capability gaps across `context-hub` (`chub`), Meta-MCP "Code Mode", standalone `SKILL.md` packages, and consolidated `references/*.md` files.

Stop and Think: Why Do Zero-Shot Self-Generated Skills Score Lower Than No Skills at All?

When an LLM is prompted to "write a reusable skill before solving this problem" without first observing a concrete failure trace, it crystallizes its own pretraining priors and unit-conversion guesses into authoritative-looking imperatives (`SkillsBench`: **−8.1 pp on Claude Code**, **−11.5 pp on Gemini CLI** [16]). Only author a `SKILL.md` from verified zero-shot failure deltas, and prove positive lift via paired `with_skill` vs. `without_skill` trials in `eval-viewer`.

System Design Invariant & Tie-Breaker

Student Lab Guide for Chapter 4: Completing Lab 06 (`TaskFlow` Platform)

- **Part A — Multi-File `agentskills.io` Validator & `chub` Grounding (`labs/lab_06_skills_chub_and_meta_mcp_code_mode`):** Inspect `starter/naive_crud_mcp_and_bloated_skill.py` and implement `work/skill_and_mcp_harness.py` (`REQ-0601..0606`, `test_req0601..0606`) to validate Level 1–3 progressive-disclosure skills (`.agents/skills/` & `.claude/skills/`) and build the versioned API documentation store supporting `chub annotate` notes wrapped in `[UNTRUSTED_LOCAL_ANNOTATION]` guards.
- **Part B — Meta-MCP "Code Mode" (`search` + `execute`) & `SkillsBench` Ablation:** Replace the 120-endpoint CRUD tool catalog with dynamic `mcp__meta__search` + sandboxed `mcp__meta__execute` meta-tools, verify **≥ 90% schema token reduction**, evaluate paired `with_skill` vs. `without_skill` ablation deltas, and run `./labs/lab_06_skills_chub_and_meta_mcp_code_mode/self_diagnose.sh`.

Benchmark / Mechanism	**Optimal Regime**	**Degraded / Trap Regime**	**Verified Engineering Rule (Lab 06)**
`SkillsBench` **Authoring** (Table 6 [16]) (`33.9% -> 50.5%`; Gemini **+24.8 pp**)	**Curated Skills: +16.6 pp** **Self-Generated: −8.1 to −11.5 pp** below baseline	Never let an LLM self-generate skills zero-shot; encode only observed baseline failure deltas.	
Skill Count & Length (Tables 8–9 [16])	`2-3` **Skills (+19.0 pp)**; `Standard` length (**+21.5 pp**)	`>= 4` **Skills (+10.1 pp)** or > 10 skills; `Exhaustive` (**+0.7 pp**)	Consolidate micro-skills into `references/*.md` and keep `SKILL.md` ≤ 500 lines.
MCP Tool Architecture (Meta-MCP [9]) (> 90% schema token savings)	**Meta-MCP** `search+execute` 110–150 CRUD REST tools ($> 10{,}000$ JSONL lines)	Expose outcome tools or Meta-MCP "Code Mode" with strict Pydantic schemas.	
API Grounding & SDT (`chub` & `eval-viewer`)	`chub get` + `chub annotate`; SDT $d' > 2.0$, $\lvert c\rvert \leq 0.25$	Unversioned web scraping; optimizing Hit Rate alone	Persist session API quirks via `chub annotate` and verify 11/11 passes in `benchmark.json`.

Table 4: Empirical benchmarks across `SkillsBench` [16], Meta-MCP "Code Mode" [9], `context-hub` [17], and SDT trigger calibration.

5 Phase 2 (Modules 7 & 8): Subagents as Context Firewalls, LLM Councils, 4-Tier Governance, Autoresearch Ratchets & Program Design

5.1 Module 7: Subagents as Context Firewalls, Multi-Agent PR Triage, llm-council & Agentic VCS (git worktree vs. Jujutsu jj)

Why Subagents Exist: Context Firewalls (50k → 500 Tokens), Not Anthropomorphic Roleplay. In multi-step agentic workflows, engineers choose among three execution scopes: **Conversational Prompts** (interactive co-design), **One-Off Bounded Prompts** (self-contained single-file diffs), and **Isolated Subagents** [5, 12]. Subagents do not exist to roleplay corporate job titles; they exist as **Context Firewalls**. If a parent agent reads 30 repository files (50,000 tokens) to locate an authentication bug, its own context window crosses the 40% Dumb Zone before coding even begins. Delegating discovery to a child subagent burns those 50,000 search tokens inside an ephemeral child window and returns a distilled **500-token summary with exact `file:line` citations**, preserving 99% of the parent's Smart Zone capacity [12].

The Seven-Phase `feature-dev` Pipeline and Multi-Agent Pull-Request Triage. Both Google Antigravity (`.agents/agents/*.md` for `code-explorer`, `code-architect`, and `code-reviewer`, plus `.agents/workflows/feature-dev.md`) and Claude Code (`.claude/agents/*.md` + `/feature-dev`) orchestrate specialized subagents across a **7-Phase Pipeline** [10, 11]: (1) *Discovery*, (2) *Parallel Read-Only Exploration (`code-explorer`, restricted to `Read`, `Grep`, `Glob`)*, (3) *Mandatory Clarifying Questions Gate (`ClarificationGateError` halt)*, (4) *Tri-Lens Architecture (`code-architect`: `minimal_changes`, `clean_architecture`, and `pragmatic_balance`)*, (5) *Implementation*, (6) *Multi-Agent Pull-Request Review (`code-reviewer`)*, and (7) *Summary*. During Phase 6, the harness spawns **4–5 parallel review subagents** (Security, Performance, Architecture, Style, and Edge Cases), suppresses low-confidence nitpicks via a strict **'confidence >= 80'** gate, and classifies findings into **'Must Fix [Red] / Recommended [Yellow] / Consider [Green]'** [9] (Figure 10).

Three-Stage Anonymized Cross-Model Peer Review (`llm-council`) and Chairman Synthesis. When evaluating high-stakes architectural designs or subtle concurrency bugs, relying on a single model family risks correlated blind spots and self-preference bias. The open-source `llm-council` architecture [18] implements a three-stage multi-model verification protocol (Figure 10): **Stage 1 (Independent First Opinions)** dispatches the engineering prompt in parallel across diverse frontier models; **Stage 2 (Anonymized Cross-Model Peer Review)** strips model identities (`Response A`, `Response B`, `Response C`) and asks each model to evaluate and rank its peers without bias; and **Stage 3 (Chairman Synthesis)** aggregates the peer rankings and critiques into a final adjudicated engineering decision.

Parallel Agent Workspace Isolation and Stacked Change Management: `git worktree` vs. Jujutsu (`jj`). When 3–10 parallel coding agents execute concurrently across four cold-start stages (*object acquisition*, *working-copy materialization*, *environment hydration*, and *agent orientation*), version control faces two distinct cost equations [19]: $\text{Cost}_{\text{prov}} = \text{Starts} \times t_{\text{cold}} \times c_{\text{runner}}$ vs. $\text{Cost}_{\text{integ}} = \text{Changes} \times t_{\text{merge}} \times c_{\text{eng}}$. Native **'git worktree'** (Antigravity 2.0 Mission Control and `claude --worktree` / `isolation: worktree`) paired with a shared object cache or `git clone --filter=blob:none` slashes $\text{Cost}_{\text{prov}}$ while preserving Git LFS, submodules, and `.gitattributes`. However, when parallel agents continuously rewrite stacked commits or collide on shared files, Git's stateful index and abortive rebase (`DETACHED HEAD` / `.git/rebase-merge`) freeze unattended agent loops. Colocating open-source **Jujutsu (`jj`)** [19] via `jj git init --colocate` and `jj workspace add ../agent-2` slashes $\text{Cost}_{\text{integ}}$ through four core invariants:

1. **Working Copy Is a Commit (`@`) & Stable Change IDs (`k-z`):** Every `jj` command auto-snapshots filesystem edits into `@` without `git add/stash`, while stable **Change IDs** survive rewrites (`jj new <peer_change_id>`, `jj workspace update-stale`).

2. **First-Class Conflict Algebra ($\Delta M = \text{Tree}(M) - \text{AutoMerge}(P_1, P_2)$) — Rebases Never Fail:** Conflicts store as algebraic trees ($A + (C - B)$) inside commits so rebases never abort mid-loop, descendants auto-rebase (`auto-evolve`) immediately, and N-parent integration stacks (`jj new changeA changeB`) rebase deterministically without "evil merge" corruption (`jj log -r "roots(mutable() & conflicts())" --no-graph`).
3. **Transactional Operation Log & Non-Interactive Reshaping:** Every repository mutation is journaled for instant `jj op log` / `jj undo` / `jj op restore <OP_ID>` rollback, paired with non-interactive `jj split -m "..." path`, `jj squash --into <REV> -u`, `jj absorb`, and `jj converge --no-interactive`.
4. **Pragmatic Hybrid Rule (`git worktree` + Colocated `jj`):** Never interleave mutating `git` and `jj` commands; use pure `git worktree` when Git LFS, submodules, or `.gitattributes` are required, and enforce disjoint module ownership.

Figure 5.1 Blueprint: Subagent Firewalls, PR Triage, `llm-council` & `git worktree` / Jujutsu (`jj`)

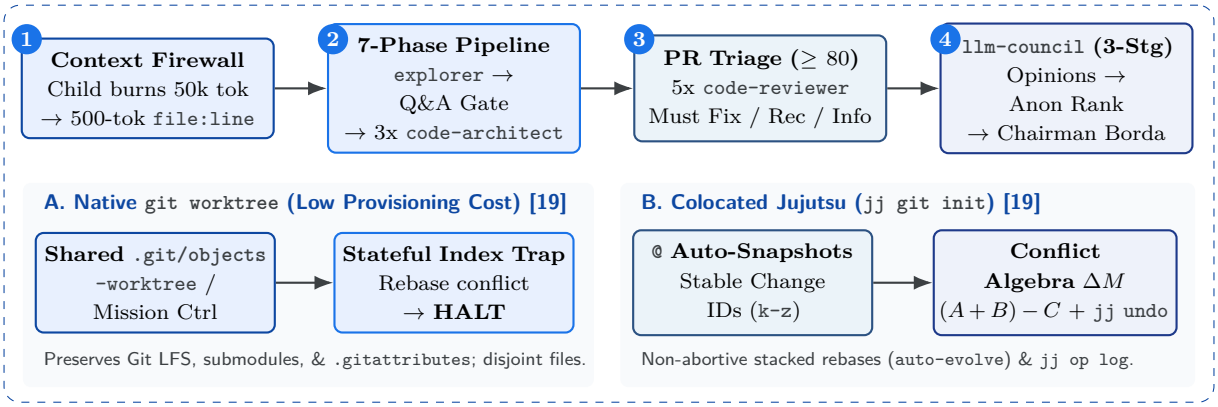


Figure 10: Layer 3 Multi-Agent Orchestration: subagent context firewalls (50k → 500 tokens [12]), 7-phase `feature-dev` [10, 11], multi-agent PR review triage [9], 3-stage anonymized `llm-council` [18], and `git worktree` vs. colocated Jujutsu (`jj`) workspace isolation [19].

5.2 Module 8: 4-Tier Governance, Remediation Linters, `Autoresearch` Ratchets & Program Design

Four-Tier "Governed by Design" Harnesses (`openworker`) and Remediation-Aware Linters. Production agent plugins (`.agents/plugins/<name>/plugin.json` on `agy` and `.claude-plugin/plugin.json` on `claude`) combine lifecycle hooks (`PreToolUse`, `PostToolUse`, `Stop`) with four defensive tiers [14, 17]:

1. **Tier 1 — Hard Floors (`PreToolUse` Exit Code `2`):** Irreversible or policy-violating actions (editing locked `adversarial_tests/` or `prepare.py`, running destructive shell commands) are blocked deterministically with Exit Code 2 (*"the fixer is never the only checker"*—combining deterministic AST/`semgrep` scanners with model review).
2. **Tier 2 — Ladder of Earned Autonomy & Reviewer Circuit Breakers:** One-off approvals graduate into scoped allowlists, while unattended loops route actions through an independent reviewer model whose circuit breaker trips on repeated denials.
3. **Tier 3 — Provenance Audit Trail:** Every tool call records `auto-approved`, `user-approved`, or `denied` with timestamped reviewer rationale.
4. **Tier 4 — OS/Container Sandbox (`NVIDIA OpenShell`) & Remediation-Aware Linters:** Commands execute inside an isolated container where API keys stay outside the wall, while `PostToolUse` structural linters emit agent-actionable error recipes: `[VIOLATION] -> [WHY] -> [HOW TO FIX]` [14].

Bounded Autonomous Ratchets (`autoresearch`), The Simplicity Criterion & Recursive Self-Improvement. When an engineering domain possesses a fast, scalar objective function—such as minimizing validation bits-per-byte (`val_bpb`) in `autoresearch` (≈ 12 experiments/hr, 83 overnight experiments yielding 15 kept improvements from `0.9979` down to `0.9773` in `progress.png` [18]) or achieving a **52× CPU training kernel speedup** in an 8-hour autonomous loop vs. a 4× human baseline [7]—teams deploy a **Bounded Autonomous Ratchet** (Figure 11). Empirical recursive self-improvement (RSI) telemetry [7] documents three structural shifts: (1) **Code Contributed per Person by Quarter** stayed flat at **1.0**× across 2021–2024 before surging to **2.5**× (Q4 '25), **5.8**× (Q1 '26), and **8.0**× (Q2 '26 Mythos Preview); (2) **Claude Code Session Success Rate** on open-ended ML research tasks climbed from **26%** (Sep '25, dipping to **11%** in Oct '25) to **71%** (Apr '26), **76%** (May '26), and **91%** (Sep '26), converging with trivial/routine/substantial tiers at **88%–92%**; and (3) on **$n = 129$ historical researcher dead-ends**, blind peer review showed the model choosing a strictly better next step than the human researcher **64%** of the time (**+9% tie = 73%**, up from **22%** with Haiku 3 and **51%** with Opus 4.5, surpassing the **50%** human baseline toward the **90%** ceiling):

- **Three-File Harness Separation:** `prepare.py` (*Immutable Evaluation Sensor*—read-only constants and `evaluate_bpb` protected by a Hard Floor hook), `train.py` (*Bounded Mutable Target*—the single file the agent edits), and `program.md` (*Human-Authored "Research Org Code"*) [18].
- **The Simplicity Criterion & Git Keep-or-Revert Ratchet:** Every 5-minute experiment commits a candidate diff, redirects logs (`> run.log 2>&1`), and evaluates both metric improvement and code complexity (*"A 0.001 val_bpb improvement that adds 20 lines of hacky code? Probably not worth it. A 0.001 improvement from deleting code? Definitely keep."*). Winning commits advance the branch; regressions or crashes trigger an immediate `git reset --hard`.

Durable Execution (`Temporal`, `LangGraph`, `Restate`, `DBOS`, `Inngest`) & The Meta-Harness Layer (`Omnigent`). As autonomous agent horizons expanded from 4 minutes in 2024 to **12–16 hours** in 2026, fragile in-memory Python `while` loops and single-harness silos hit two scaling limits: first, an LLM `429` rate-limit, MCP tool timeout, or container restart at turn 18 wipes in-memory state; second, engineers running 4–5 specialized harnesses at once (`claude`, `agy`, `Codex`, `Pi`) lack cross-harness composition, stateful governance, and multiplayer sharing. Production platforms solve both via **Durable Execution** and the **Meta-Harness Layer** [20]:

- **Durable Execution (`Temporal`, `LangGraph`, `DBOS`, `Restate`, `Inngest`):** In durable agent frameworks, the **deterministic Agent Loop (`@workflow.defn`)** is separated from **non-deterministic LLM and MCP calls (`@activity.defn`)**. Every completed activity output is appended to a persistent event history so crashed workers replay prior turns without re-billing LLM calls, while `workflow.sleep()` and external **Signals** pause multi-day Human-in-the-Loop (HITL) approvals with zero compute cost.
- **The Meta-Harness Layer (`Databricks Omnigent`):** Just as Kubernetes lifted fleet management above individual servers, `Omnigent` [20] sits *above* individual coding harnesses (`Claude Code`, `Codex`, `Pi`, `OpenAI/Claude Agents SDK`) behind a universal *messages/files-in, text-streams/tool-calls-out* runner across four architectural layers (*Heterogeneous Agents → Omnigent Runner → Omnigent Server [History, Catalog, Policies, MCPs, Artifacts, Skills + PostgreSQL/MLflow + GEPA/MemEx/RLM] → 5 Surfaces [TUI, Web UI with inline diff comments, Native App, Mobile UI, REST API]*): **(1) Composition (*Combine*)**, **(2) Stateful Contextual Control (*Control*)** (escalating `git push` after `npm install`, pausing every $100 spend, and **Egress-Proxy Credential Injection** outside the sandbox), and **(3) Multiplayer Collaboration (*Share*)**.

Why Unattended Software Factories Fail and Four-Phase Front-Loaded Program Design. Why do unattended "lights-off / dark software factories" succeed on bounded numeric kernels yet collapse into unmaintainable spaghetti after 8 weeks of building enterprise products? The root cause is the **RLVR Reward Horizon Gap** [20]: Reinforcement Learning from Verifiable Rewards trains models to maximize $t = 0$ immediate test passes, whereas software architecture is evaluated at $t = 6$ months when requirements

evolve. Deterministic harnesses **raise the floor** (catching syntax, lint, and local test regressions) but cannot raise the architectural ceiling. Building durable systems requires keeping human engineers "on the loop" via **4-Phase Front-Loaded Alignment** (locking types, interfaces, and call-trees in Phase 3 before coding):

1. Product → 2. Architecture → **3. Program Design (Types & Call-Trees)** → 4. Vertical Slices (1)

Figure 5.2 Blueprint: `Autoresearch` (0.9979→0.9773), `Omnigent` & Anthropic RSI Charts

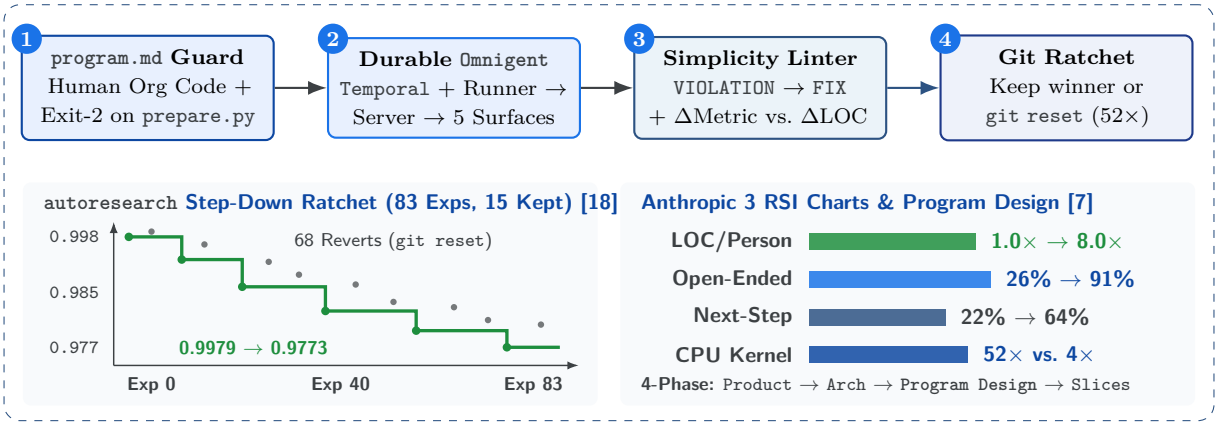


Figure 11: Layer 4 Autonomous Loops & Meta-Harnesses: the `autoresearch` 83-experiment step-down ratchet (`0.9979` → `0.9773` [18]), Databricks `Omnigent` [20], and Anthropic's 3 RSI charts (**8.0**× quarterly LOC/person, **26%→91%** open-ended success, and **64%** next-step win rate [7]).

Common Misconception vs. Engineering Reality: Unattended "Lights-Off" Software Factories Can Replace Human Program Design

Common Misconception: Because an autonomous ratchet loop (`autoresearch`) can optimize a single-file numeric kernel overnight, an unattended "dark software factory" can build and evolve a multi-service enterprise product without human intervention.

Engineering Reality: Due to the **RLVR Reward Horizon Gap** [20], models optimize for $t = 0$ test passes rather than $t =$ 6-month domain extensibility. Harnesses raise the floor (blocking syntax, lint, and test regressions) but cannot raise the architectural ceiling—human engineers must lock **Phase 3 Program Design** (domain types, module boundaries, and call-trees) before delegating vertical slices.

System Design Invariant & Tie-Breaker

Student Lab Guide for Chapter 5: Completing Labs 07 & 08 (`TaskFlow` Platform)

1. **Lab 07 (`labs/lab_07_subagent_firewalls_and_peer_council`) — Context-Firewall Sub-agents, Peer-Review Council & Workspace Isolation:** Inspect `starter/noisy_roleplay_subagents.py` and implement `work/subagent_council.py` (`REQ-0701..0706`, `test_req0701..0706`) to enforce read-only `code-explorer` (50k → 500-token compression), multi-agent `Must Fix / Recommended / Consider` PR triage (`confidence >= 80`), 3-stage anonymized `llm-council` Borda synthesis, and disjoint `git worktree` / colocated Jujutsu (`jj`) workspace isolation (`./labs/lab_07_subagent_firewalls_and_peer_council/self_diagnose.sh`).
2. **Lab 08 (`labs/lab_08_governance_linters_and_autoresearch_ratchet`) — 4-Tier Governance Hooks, Remediation Linters & The `Autoresearch` Ratchet:** Inspect `starter/dark_factory_reward_hacker.py` and implement `work/autoresearch_harness.py`

Phase 2 Mechanism	Architectural Pattern	Core Invariant / Protocol	Implementation in Labs 07–08
Context-Firewall Subagents	Ephemeral child window	Burn 50k search tokens in child window; return 500-token `file:line` digest.	`code-explorer`, `code-architect`, `code-reviewer` in `.agents/agents/` & `.claude/agents/`.
Multi-Agent PR Triage & `llm-council`	5-lens review & Borda peer rank	Filter review items at confidence ≥ 80; Stage 2 anonymized peer ranking.	Synthesizes `Must Fix / Recommended / Consider` + Chairman verdict (Lab 07).
Agentic VCS: `git worktree` **vs. Jujutsu (**`jj`**)**	Parallel checkout & conflict algebra	`git worktree` cuts Provisioning Cost; colocated `jj` (`@` commit, Change IDs, Conflict Algebra ΔM, `jj op log`/`jj undo`) cuts Integration Cost.	Disjoint worktree / `jj workspace` ownership & conflict-safe stacked diffs (Lab 07).
4-Tier Governed Harness	`openworker` & `NVIDIA OpenShell`	Hard Floors (`PreToolUse` Exit 2), Earned Autonomy, Provenance Log, Sandbox.	Blocks edits to `adversarial_tests/` or `prepare.py` and logs provenance (Lab 08).
Remediation Linters	Self-repairing feedback sensor	Format linter errors as `[VIOLATION] -> [WHY] -> [HOW TO FIX]`.	`PostToolUse` architectural boundary linter for autonomous self-repair (Lab 08).
Durable Execution & Meta-Harnesses	`Temporal`, `LangGraph`, `DBOS`, `Restate`, `Omnigent`	Event-history activity replay + `Omnigent` YAML composition, stateful taint/cost policies, & egress-proxy secrets.	Crash-recoverable long-horizon agent checkpoints & policy ledgers (Lab 08).
`Autoresearch` **Ratchet & Program Design**	Keep-or-revert loop & 4-Phase Design	Simplicity Criterion (Δmetric vs. ΔLOC) + 4-Phase Program Design types/call-trees.	Git Keep-or-Revert Ratchet with circuit breaker + Program Design verifier (Lab 08).

Table 5: Complete Phase 2 Harness & Meta-Harness Engineering reference matrix covering Modules 7 and 8 (Labs 07–08).

(REQ-0801..0806, test_req0801..0806) to enforce a PreToolUse Exit-Code-2 hard-floor gate, a [VIOLATION] -> [WHY] -> [HOW TO FIX] structural linter, a 4-Phase **Program Design** type/call-tree verifier, and a Simplicity-Criterion autoresearch keep-or-revert ratchet loop (./labs/lab_08_governance_linters_and_autoresearch_ratchet/self_diagnose.sh).

3. **Full-Course Self-Diagnosis & 3-Rung Self-Check Hints:** Every lab's WALKTHROUGH.md and docs/interactive_visual_guide.html include an **Everyday Metaphor (Known → Target)**, a **Spiral Retrieval Bridge** across the unified TaskFlow workspace, and a **3-Rung Graduated Hint Ladder** (*Spotlight* → $N = 1$ *Toy Case* → *Full Walkthrough*). Run ./self_diagnose_all.sh from the repository root to verify all 8 labs and 96 unit/adversarial tests in under 1 second.

5.3 References & Further Reading

[1] U. Joshi & M. Fowler, *"What Is Code?"* (2026).

[2] L. Wittgenstein, *Tractatus Logico-Philosophicus* (1921) & *Philosophical Investigations* (1953); T. Winograd & F. Flores, *Understanding Computers and Cognition* (Addison-Wesley, 1986).

[3] S. I. Wang, P. Liang & C. D. Manning, *"Learning Language Games through Interaction"* (*ACL*, 2016).

[4] P. Molino & J. Tagliabue, *"Wittgenstein and Large Language Models"* (2023).

[5] A. Ng & DeepLearning.AI, *The AI Engineering Skills Map: Four Pillars and Twenty Core Competencies* (2026).

[6] A. Karpathy, *"Vibe Coding"* & *"Software 3.0: Programming in the Era of the LLM Operating System"* (2025).

[7] J. Clark & M. Favaro, *"When AI Builds Itself: Recursive Self-Improvement and Autonomous Horizons"* (Anthropic Institute, 2026).

[8] P. Byrne, *"The Model Behind the Answer Now Changes Every 17 Days"* (SearchIntel Research, 2026).

[9] M. Eric, *CS146S: The Modern Software Developer — Lecture Notes on Harnesses, RePPIT, and Meta-MCP* (Stanford University, 2025–2026).

[10] Google Developers, *Getting Started with Google Antigravity*, *Hands-on with Antigravity CLI*, & *Build and Deploy to Google Cloud with Antigravity* (2025–2026).

[11] Anthropic Academy, *Claude Code 101* & *Effective Context Engineering for AI Agents* (2025–2026).

[12] D. Horthy, *"No Vibes Allowed: Solving Hard Problems in Complex Codebases"* (HumanLayer, 2025).

[13] B. Böckeler & K. Morris, *"Harness Engineering: Cybernetic Guides and Sensors for Coding Agents"* (*Exploring Gen AI*, 2026).

[14] OpenAI Engineering, *"Harness Engineering: Leveraging Codex in an Agent-First World"* (2026).

[15] Agent Skills Working Group, *Agent Skills Open Specification (SKILL.md Progressive Disclosure Standard)* (2025–2026).

[16] X. Li et al., *"SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks"* (2026).

[17] A. Ng et al., *Open-Source Agentic Engineering Suite: aisuite, openworker, context-hub (@aisuite/chub), and translation-agent* (2024–2026).

[18] A. Karpathy, *Open-Source Agentic Systems: rendergit, llm-council, and autoresearch* (2025–2026).

[19] K. Riedl, *"Git Worktrees vs. Jujutsu (jj) for Parallel AI Coding Agents"* (2026); M. von Zweigbergk, *Jujutsu VCS* (jj-vcs).

[20] M. Zaharia, M. Uhlenhuth & C. Zumar, *"Omnigent: The Meta-Harness Architecture"* (Databricks, 2026); D. Horthy, *"Harness Engineering Is Not Enough: Why Software Factories Fail"* (2026).